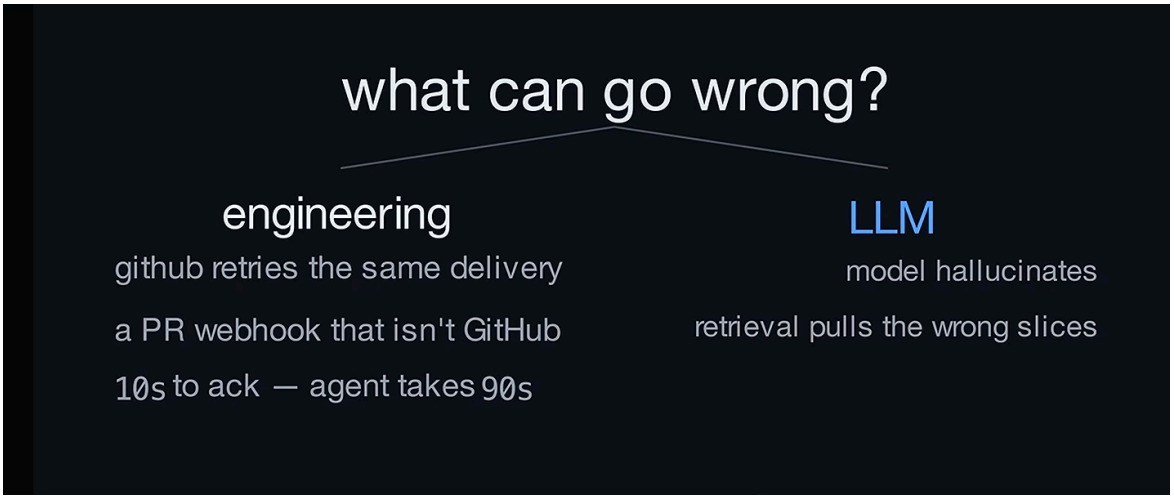
Designing and building a PR review multi agent system

Created by	b bee
Category	
Last edited by	b bee



Universal system design template

1. Map everything from scratch. for example in the RAG project, map how exactly the doctor works with the tool so you know how to automate it and make it better. how exactly they start, what they click, what they upload etc.
2.

Move 2. Name the trigger and the output

Every automatable workflow has a precise trigger and a precise output. "A claim email arrives at claims@company.com," not "claims come in." "A structured review is posted to the PR," not "code gets reviewed." If you cannot state both in one sentence each, you have not looked closely enough.

Move 3. Assign components

Take each step and assign it to a component type. Detecting that work is needed is a **trigger**. Fetching data is a **tool/API**. Reading unstructured input or writing language is an **LLM**. A score or classification that must be identical every time is **deterministic ML**. A judgment with legal, financial, or safety stakes is a **human checkpoint**.

Move 4. Choose autonomy

Decide how much the system does on its own versus how much it defers to a human. This is not a default — it is a design choice driven by the consequence of error. Section 0.3 gives the full spectrum.

Move 5. Design for failure

Walk each component and break it on purpose. What happens when the LLM hallucinates, the API times out, two parallel agents conflict? Every component should fail gracefully.

``Full autonomy cannot be given to systems with health or money, 100 % autonomy is never allowed.

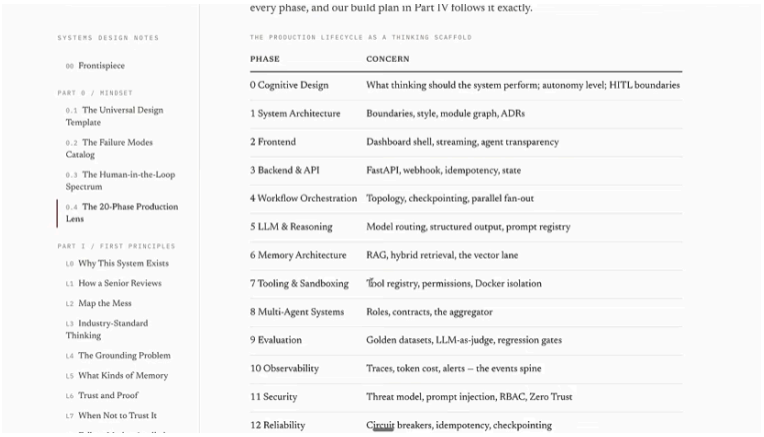
SEVEN GENERAL AGENTIC FAILURE MODES

MODE	WHAT HAPPENS	DESIGN AGAINST IT WITH
Hallucination in a critical path	The model states something plausible but false in a place that matters	Citation requirement, a fact-check layer, human review for high stakes, prompts that permit "I don't know"
Model drift	Accurate at deployment, degrades as the world changes	Monitoring dashboard, alert thresholds, periodic retraining, rules fallback
Tool / API timeout	An external system stalls and the pipeline hangs	Timeout-and-retry, graceful degradation on partial data, circuit breaker on a dead service
Feedback-loop poisoning	Bad feedback is stored and degrades future behavior	Minimum evidence threshold, audit for protected proxies, decay on old feedback, human reset
Orchestration deadlock	Two parallel steps wait on each other; a merge never receives an input	Timeouts on every step, health checks, idempotent operations, dead-letter queue
Human bottleneck	Auto-handling works, but the escalation queue grows faster than humans clear it	Escalation-rate monitoring, queue prioritization, threshold tuning under load, honest capacity planning
The "almost-right" problem	Output 90% correct, 10% subtly wrong, while reviewers drift into complacency	Rotate reviewers, flag low-confidence outputs, random audits, inject known-wrong inputs to test vigilance

0.3 The Human-in-the-Loop Spectrum

Not every system needs the same level of human involvement. The right level is a design choice, not a default. Five levels, and the three factors that pick one.

LEVEL	DESCRIPTION	WHERE IT FITS
Full automation	System handles everything; human samples periodically	Routine, low-stakes, reversible work
Human reviews output	System produces, human verifies before it goes out	Drafts with reputational stakes
Human handles exceptions	System auto-handles easy cases, human sees the hard ones	Anomalies, low-confidence cases, escalations
Human decides, system prepares	System gathers all context, human makes the call	High-consequence, irreversible decisions
Full human with AI assist	Human does the work, system helps at specific steps	Early-stage, low-trust, or creative work

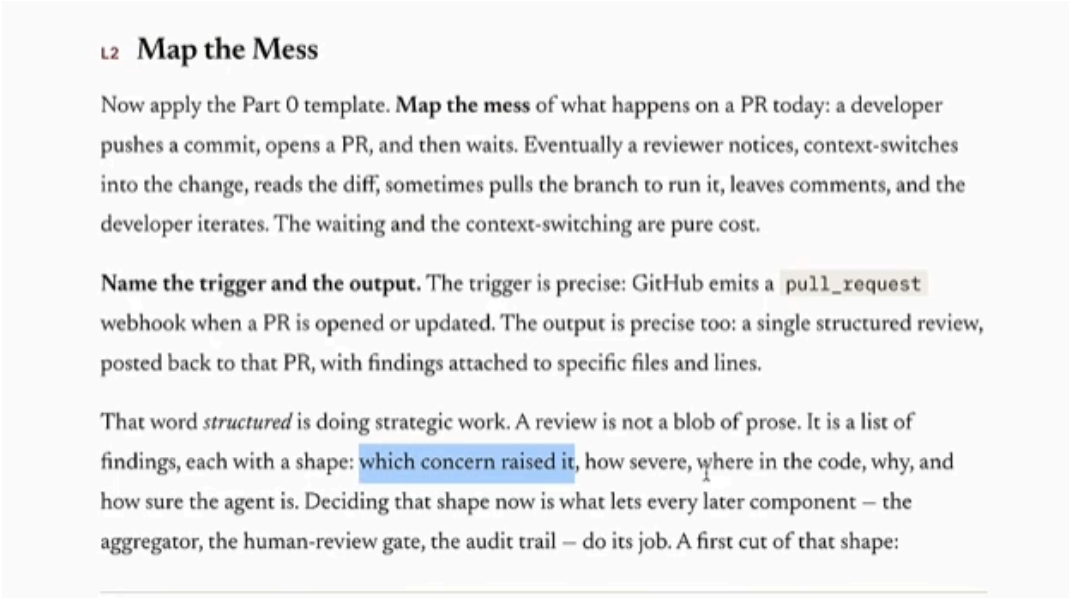


L1 Start From How a Senior Reviews

Before touching LLMs, prompts, or vector databases, ask how a senior engineer actually reviews a pull request. When you are designing something genuinely hard, look for a system that already solved it. Human review has been refined over decades, and the structure we need is sitting inside how a good reviewer thinks.

Watch one closely. They do four things that a naive single-prompt reviewer does not. They **bring codebase context** — they know this function overrides a base class, that this pattern contradicts a past decision. They **reason across separate concerns** — a security pass, a correctness pass, a test-coverage pass, a documentation pass, each with a different mindset. They **stay skeptical** — they do not assume the diff is correct. And they **cite evidence** — "this is wrong because line 40 can be null here," not "looks off."

Read those four again as an engineer. "Brings codebase context" means the agent needs retrieval. "Reasons across separate concerns" means the agent is not one reasoner but several. "Stays skeptical" and "cites evidence" mean every finding needs a rationale and a confidence. The way a senior reviews is not trivia — it is a decomposition waiting to be built.



L2

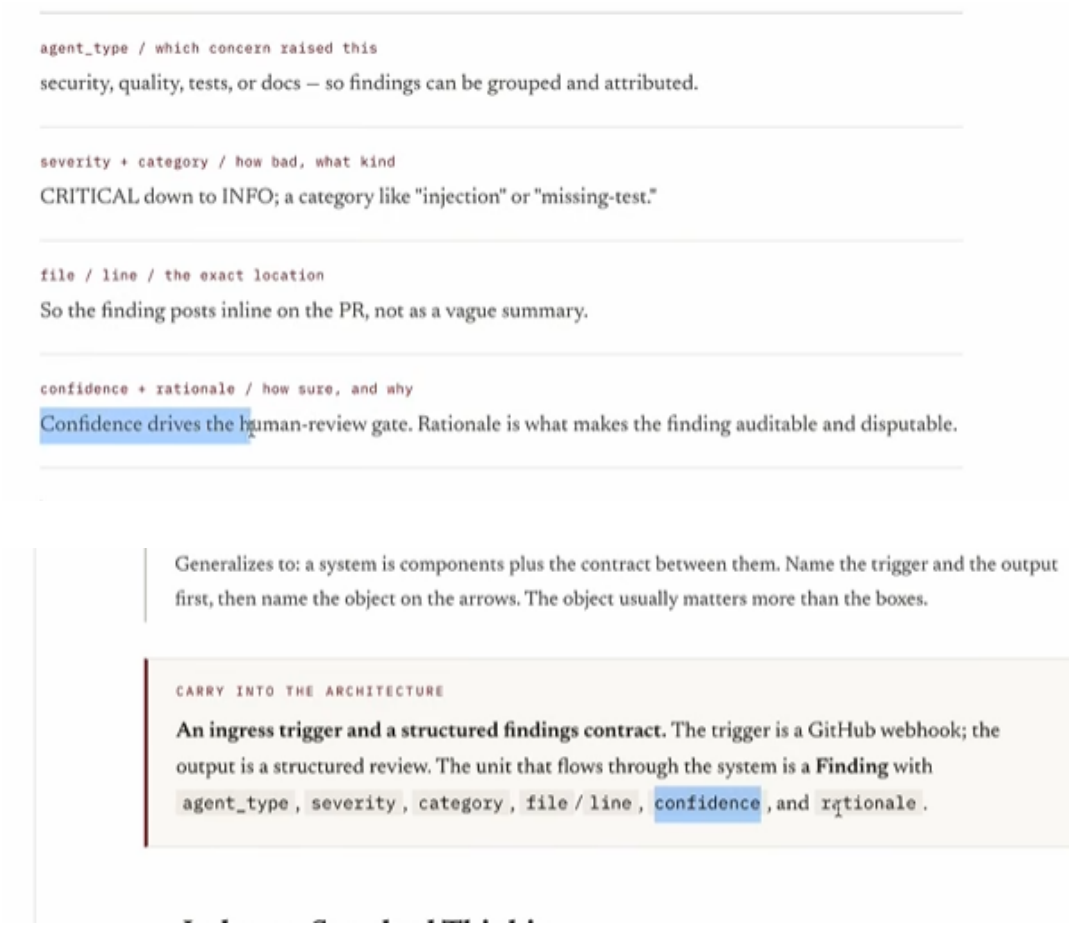
Map the Mess

Now apply the Part 0 template. **Map the mess** of what happens on a PR today: a developer pushes a commit, opens a PR, and then waits. Eventually a reviewer notices, context-switches into the change, reads the diff, sometimes pulls the branch to run it, leaves comments, and the developer iterates. The waiting and the context-switching are pure cost.

Name the trigger and the output. The trigger is precise: GitHub emits a `pull_request` webhook when a PR is opened or updated. The output is precise too: a single structured review, posted back to that PR, with findings attached to specific files and lines.

That word *structured* is doing strategic work. A review is not a blob of prose. It is a list of findings, each with a shape: which concern raised it, how severe, where in the code, why, and how sure the agent is. Deciding that shape now is what lets every later component — the aggregator, the human-review gate, the audit trail — do its job. A first cut of that shape:

Example of the structure usually in PR reviews or a list of findings:



`agent_type / which concern raised this`

security, quality, tests, or docs — so findings can be grouped and attributed.

`severity + category / how bad, what kind`

CRITICAL down to INFO; a category like "injection" or "missing-test."

`file / line / the exact location`

So the finding posts inline on the PR, not as a vague summary.

`confidence + rationale / how sure, and why`

Confidence drives the human-review gate. Rationale is what makes the finding auditable and disputable.

> Generalizes to: a system is components plus the contract between them. Name the trigger and the output first, then name the object on the arrows. The object usually matters more than the boxes.

> CARRY INTO THE ARCHITECTURE
>
> **An ingress trigger and a structured findings contract.** The trigger is a GitHub webhook; the output is a structured review. The unit that flows through the system is a **Finding** with `agent_type`, `severity`, `category`, `file` / `line`, `confidence`, and `rationale`.

L4 The Grounding Problem

Here is where most single-LLM reviewers fail. An LLM handed a diff in isolation knows what changed but not what it changed *within*. It cannot know that this function overrides a base method, that this pattern was deliberately rejected in a past decision, or that the test convention in this repo is table-driven. So it does what models do under uncertainty: it guesses, confidently. That is hallucination in a critical path — the first failure mode from the 0.2 catalog.

A senior reviewer does not have this problem because they *know the repo*. The design question is: what gives the agent that knowledge? It cannot be the full repository in the prompt — that exhausts the context window on any non-trivial codebase, and most of it is irrelevant to a given diff. The answer is retrieval: for each diff, fetch only the most relevant slices of the codebase and put those in the prompt.

> **An ungrounded reviewer is a confident stranger. A grounded one is a colleague who has read the code. Retrieval is what turns the stranger into the colleague.**

L5 What Kinds of Memory Does Review Need?

L4 said "retrieve context." But context is not one kind of thing. A senior reviewer draws on several kinds of memory, and — exactly as in cognitive science — each kind has a different shape and a different access pattern. Naming them now is what determines the data design later.

KIND OF MEMORY	WHAT IT HOLDS FOR REVIEW	THE DATA SHAPE IT WANTS
Semantic	The codebase itself — functions, classes, modules, ADRs, conventions, as meaning	Vector embeddings + similarity search
Episodic	Past reviews — what was flagged before, what was disputed, what was merged	Time-stamped relational rows
Procedural	How this team likes things done — conventions, ADRs, severity policy	Small, high-priority, almost always loaded

L6 Trust and Proof

Suppose the agent posts a finding: "this endpoint is vulnerable to SQL injection, confidence 0.6." A developer disputes it. Now what? If there is no record of *why* the finding was raised — which context was retrieved, which prompt version ran, what the model returned, what it cost — the system cannot defend itself, cannot be debugged, and cannot improve. A review you cannot audit is worthless, and a system whose cost can run away unobserved is dangerous.

So trust requires a third thing beyond reasoning and grounding: **proof**. Every action the agent takes — every span of work, every LLM call, every tool call, every decision — must be recorded as an event, in time order, durably. That single stream of events is what powers three things at once: a trace viewer (reconstruct any review end-to-end), an audit trail (defend or dispute any finding), and a cost ledger (attribute every token to an agent and a model).

- For this only we talk about “Observability” and the tech stack used for this trace etc.

L7 When Not to Trust It

L0 said the system should be selective. L6 gave it the confidence field. Now apply the 0.3 HITL spectrum to *this* system. The agent is not always right, and it knows roughly when it is unsure — that is what the `confidence` on each finding measures. The design decision is what to do with that knowledge.

The answer is a confidence-weighted gate. High-confidence reviews, with no critical findings, post automatically — the agent has earned that autonomy. Low-confidence reviews route to a human approval queue. Any finding marked `CRITICAL` escalates regardless of confidence, because the consequence of a missed critical issue is too high to automate (consequence of error, from 0.3). This places the system at the "human handles exceptions" level of the spectrum, with an escalation path to "human decides."

THE CONFIDENCE-WEIGHTED GATE

CONDITION	ACTION	WHICH 0.3 FACTOR
High confidence, no CRITICAL	Post automatically	Maturity earns autonomy
Confidence below threshold	Route to human approval queue	Uncertainty, defer judgment
Any CRITICAL finding	Escalate, page a human	Consequence of error too high
Developer disputes a posted finding	Route to dispute, record feedback	Reversibility, learning loop

GENERAL FAILURE (0.2)	IN CODE REVIEW IT LOOKS LIKE	DEFENSE
Hallucination in critical path	A finding about code the agent never actually saw	Grounding (L4) + rationale + confidence (L2)
Tool / API timeout	The LLM provider or GitHub API stalls	Retries with backoff, circuit breakers
Orchestration deadlock	The aggregator waits forever on a hung agent	Timeouts on every node, dead-letter handling
The "almost-right" problem	A finding 90% right but subtly misattributed	Dedup across agents, confidence threshold, HITL (L7)
Human bottleneck	The approval queue grows faster than reviewers clear it	Escalation-rate monitoring on the events spine (L6)
Feedback-loop poisoning	The agent "learns" a wrong preference from a few disputes	Minimum evidence threshold before acting on feedback
Idempotency gap	A retried webhook posts the same review twice	Idempotency key at ingress, dedup before posting

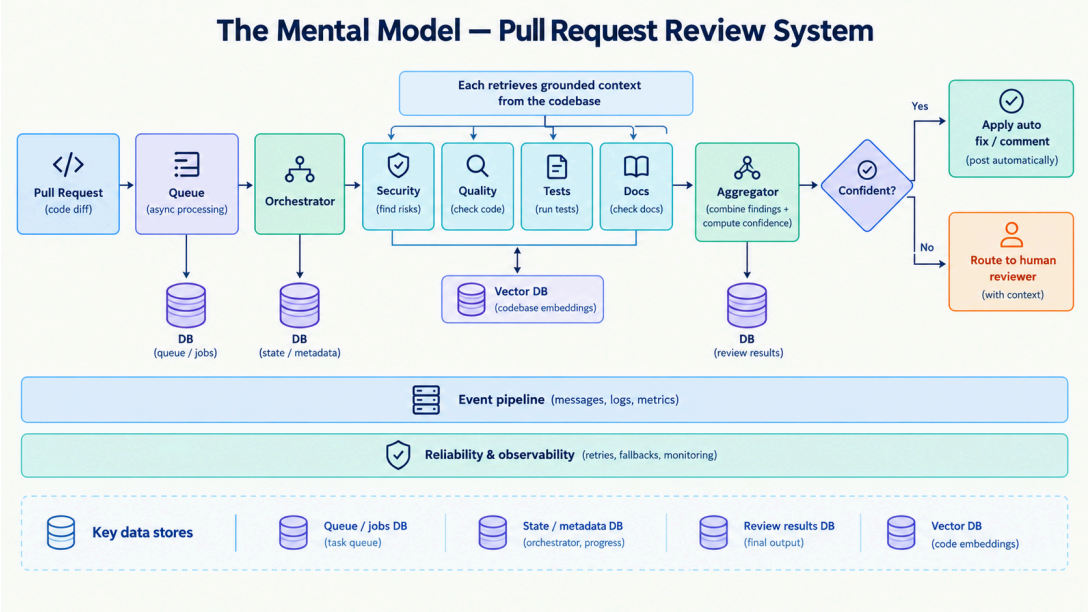
Failure analysis did not invent new parts. It justified the parts we chose and added a thin layer of reliability mechanics on top.

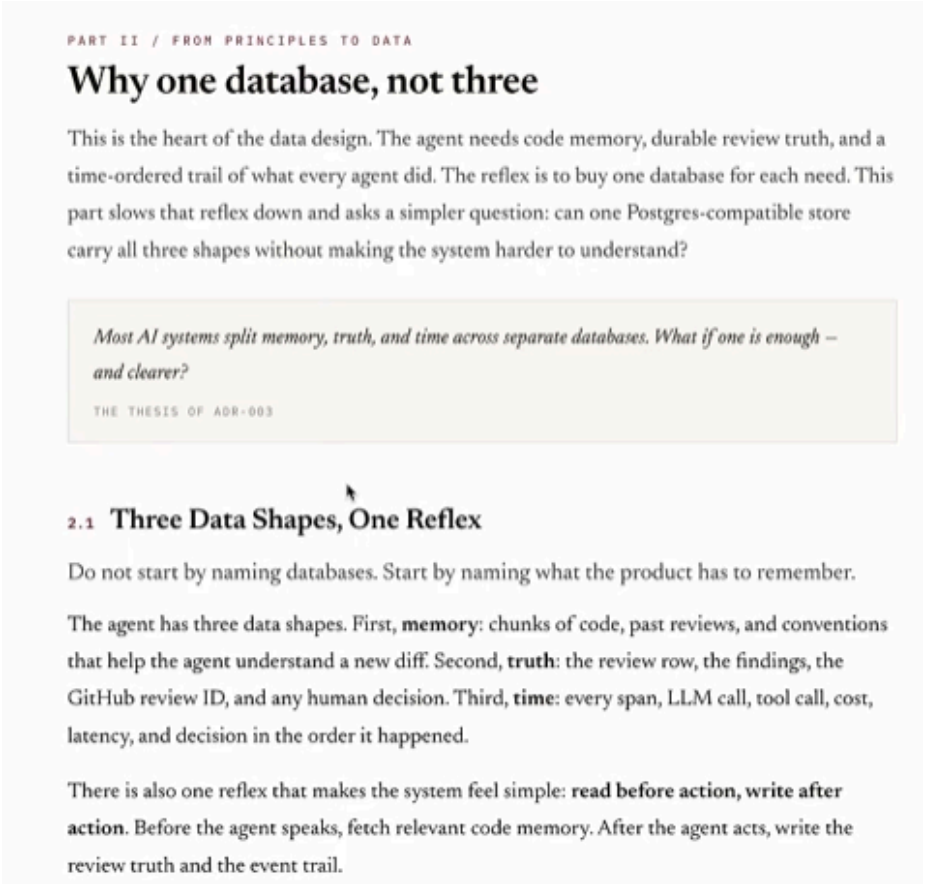
L9 The Mental Model

Before we draw a single box, the assembled reasoning in prose. The pieces we carried, in the order they were earned, already describe the whole system.

A pull request triggers the work (L2). It is enqueued, not handled inline, because the trigger must be acknowledged fast and the work decoupled from the ingress (L2, L8). An orchestrator fans the work out to four specialists — security, quality, tests, docs (L1, L3) — running in parallel. Each specialist is grounded by retrieval over the codebase (L4), because an ungrounded reviewer hallucinates. The codebase, the past reviews, and the conventions are three kinds of memory, three data shapes (L5). Each specialist returns structured findings with confidence and rationale (L2). An aggregator merges and deduplicates them, computes an overall confidence, and applies the HITL gate (L7): post automatically when confident, route to humans when not. Every action along the way is written to an events spine (L6) so the whole thing can be traced, audited, and priced. And a reliability layer (L8) keeps each step degrading to slower-but-correct.

Now ask: how many databases does this need? L5 found three data shapes; L6 added a fourth. The naive answer is four stores. Part II interrogates that answer — and arrives at one.





PART II / FROM PRINCIPLES TO DATA

Why one database, not three

This is the heart of the data design. The agent needs code memory, durable review truth, and a time-ordered trail of what every agent did. The reflex is to buy one database for each need. This part slows that reflex down and asks a simpler question: can one Postgres-compatible store carry all three shapes without making the system harder to understand?

Most AI systems split memory, truth, and time across separate databases. What if one is enough — and clearer?

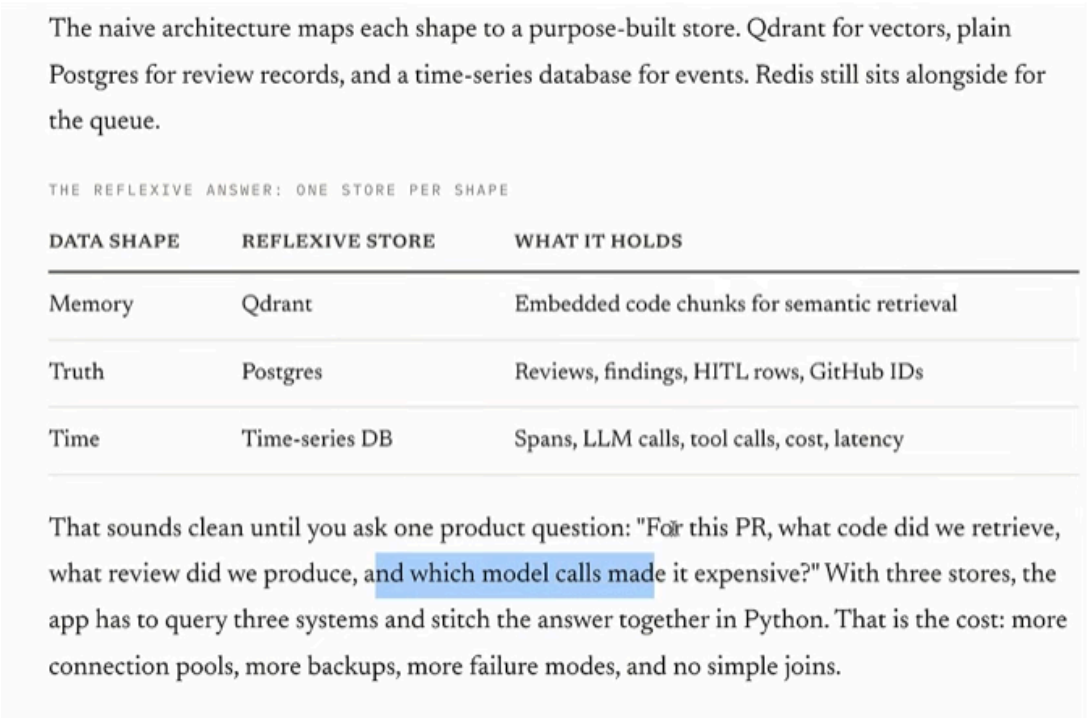
THE THESIS OF ADR-003

2.1 Three Data Shapes, One Reflex

Do not start by naming databases. Start by naming what the product has to remember.

The agent has three data shapes. First, **memory**: chunks of code, past reviews, and conventions that help the agent understand a new diff. Second, **truth**: the review row, the findings, the GitHub review ID, and any human decision. Third, **time**: every span, LLM call, tool call, cost, latency, and decision in the order it happened.

There is also one reflex that makes the system feel simple: **read before action, write after action**. Before the agent speaks, fetch relevant code memory. After the agent acts, write the review truth and the event trail.



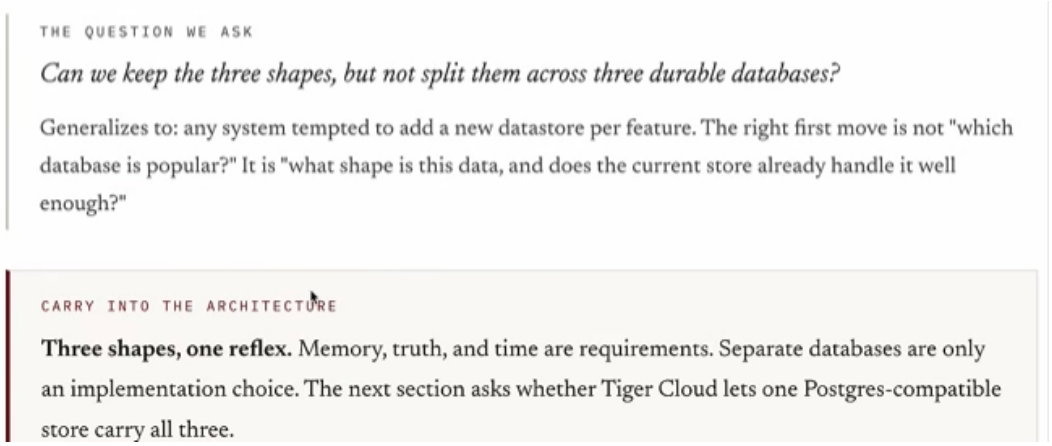
The naive architecture maps each shape to a purpose-built store. Qdrant for vectors, plain Postgres for review records, and a time-series database for events. Redis still sits alongside for the queue.

THE REFLEXIVE ANSWER: ONE STORE PER SHAPE

DATA SHAPE	REFLEXIVE STORE	WHAT IT HOLDS
Memory	Qdrant	Embedded code chunks for semantic retrieval
Truth	Postgres	Reviews, findings, HITL rows, GitHub IDs
Time	Time-series DB	Spans, LLM calls, tool calls, cost, latency

That sounds clean until you ask one product question: "For this PR, what code did we retrieve, what review did we produce, and which model calls made it expensive?" With three stores, the app has to query three systems and stitch the answer together in Python. That is the cost: more connection pools, more backups, more failure modes, and no simple joins.

This is where we make a decision as to which database can help us in storing all 3/4 types together.

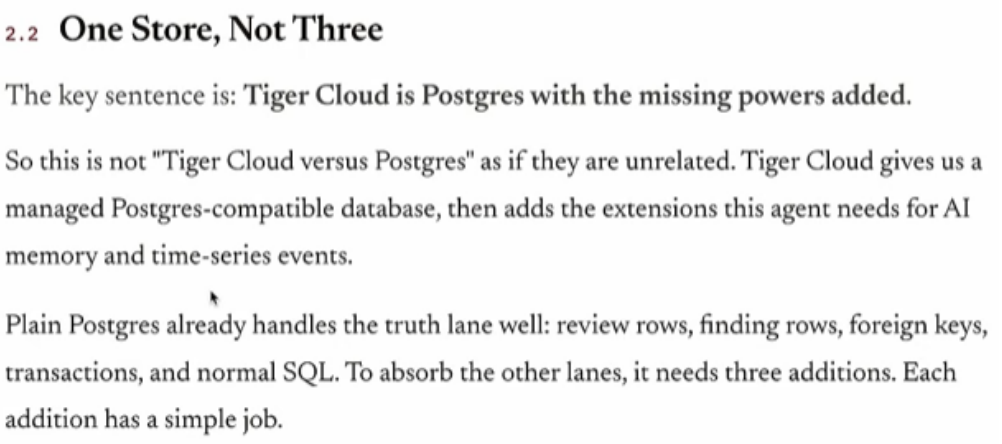


You generally don't need a separate database for every data type. Choose storage based on access patterns and consistency requirements.

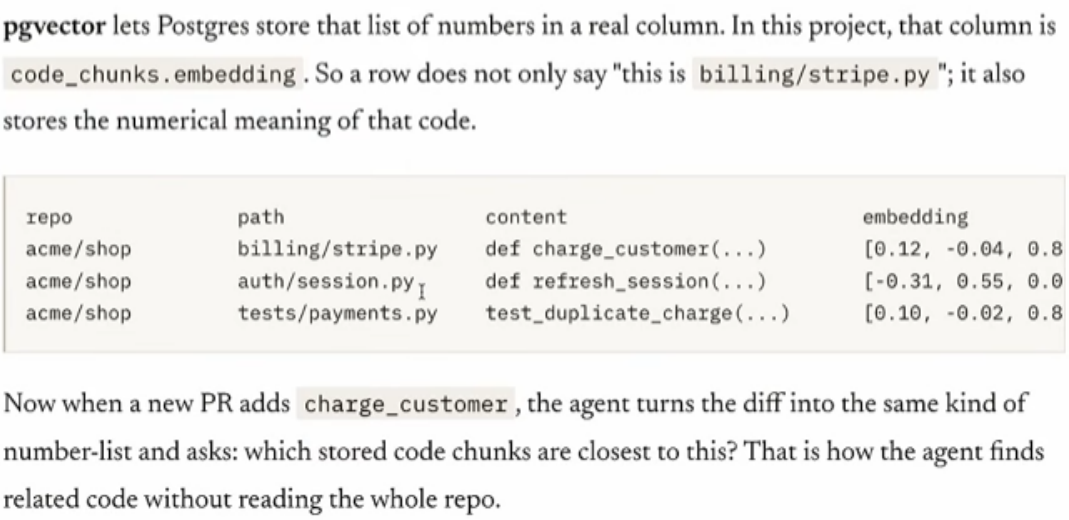
For this system, I'd start with:

- One PostgreSQL database for jobs, orchestration state, review findings, confidence scores and audit metadata. Use separate tables and JSONB where flexible fields are useful.
- A queue (e.g., SQS, RabbitMQ) for asynchronous jobs, rather than implementing a queue using database tables.
- Vector search using `pgvector` inside PostgreSQL for code embeddings, avoiding a separate vector database initially.
- Object storage (e.g., S3) only if storing large artifacts such as code snapshots or logs.

Key principle: Start with the fewest storage systems possible. Split them only when scale, performance, isolation or operational needs justify the added complexity.



basically Postgresql can already store embeddings (have to change it in my RAG project) so theres no need to have 2 different DBs to store stuff in the project.



pgvector lets Postgres store that list of numbers in a real column. In this project, that column is `code_chunks.embedding`. So a row does not only say "this is `billing/stripe.py`"; it also stores the numerical meaning of that code.

repo	path	content	embedding
acme/shop	billing/stripe.py	def charge_customer(...)	[0.12, -0.04, 0.8
acme/shop	auth/session.py	def refresh_session(...)	[-0.31, 0.55, 0.0
acme/shop	tests/payments.py	test_duplicate_charge(...)	[0.10, -0.02, 0.8

Now when a new PR adds `charge_customer`, the agent turns the diff into the same kind of number-list and asks: which stored code chunks are closest to this? That is how the agent finds related code without reading the whole repo.

2. What does each technology do?

PostgreSQL + pgvector

Think of this as a library containing all your code snippets and their numerical representations.

`pgvector` lets PostgreSQL store vectors and perform similarity searches. It can compare vectors to find which snippets are most similar to a query.

Problem: With millions of vectors, comparing a query against every single vector can become slow.

pgvectorscale

Think of this as a faster librarian who knows how to find relevant books without checking every shelf.

It extends pgvector with additional indexing and search optimizations, helping PostgreSQL find similar vectors more efficiently, particularly at larger scales.

DiskANN

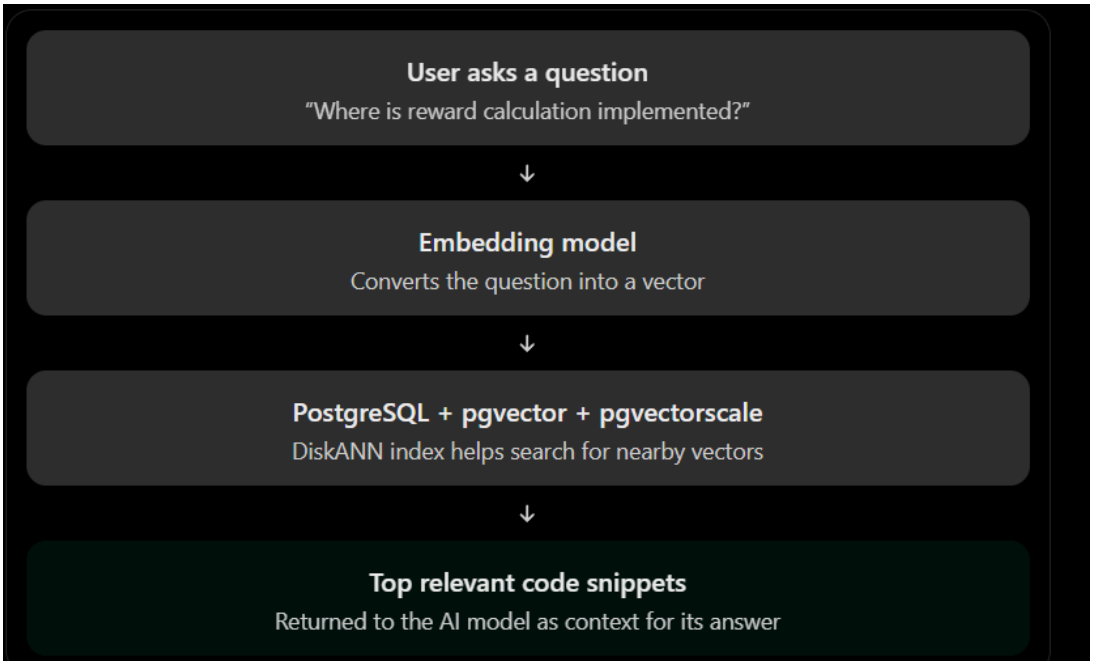
Think of this as a map of the library that helps you navigate directly toward relevant books.

DiskANN uses a graph-based search index designed to find approximate nearest neighbours efficiently, even when the index is too large to fit entirely in RAM.

Instead of keeping the entire search structure in memory, it can use SSD storage to help navigate a much larger collection without requiring enormous amounts of RAM.

3. How do they work together?

Imagine an AI coding assistant searching a codebase with 10 million code snippets.



The key benefit is that the system can search a huge collection of code without scanning every vector or keeping the entire search index in RAM.

4. What does "approximate nearest neighbours" mean?

An exact search tries to find the mathematically closest vectors by comparing against every candidate. An approximate nearest-neighbour (ANN) search uses an index to find very close matches much faster, but may miss some of the exact best matches.

This creates a trade-off:

- Exact search: More exhaustive, but potentially slower.
- ANN search: Usually faster and more scalable, but results may not be perfectly exact.

For an AI coding assistant, ANN is often a sensible trade-off because retrieving highly relevant snippets quickly matters more than guaranteeing the mathematically perfect nearest neighbours every time.

Second type of storage data:

Think of a system where AI agents review pull requests. You want to know what happened, when it happened, how long it took, and how much it cost. This is where time-series storage comes in.

1. What is a time-series table?

A time-series table stores records where time is a key part of the data.

For example, every time an AI agent performs an action, you record an event:

Timestamp	Agent	Event	Cost
09:00:01	Security	Started review	—
09:00:04	Security	LLM call	$0.018
09:00:08	Security	Finished review	—
09:00:12	Quality	Started review	—

This is useful for tracking:

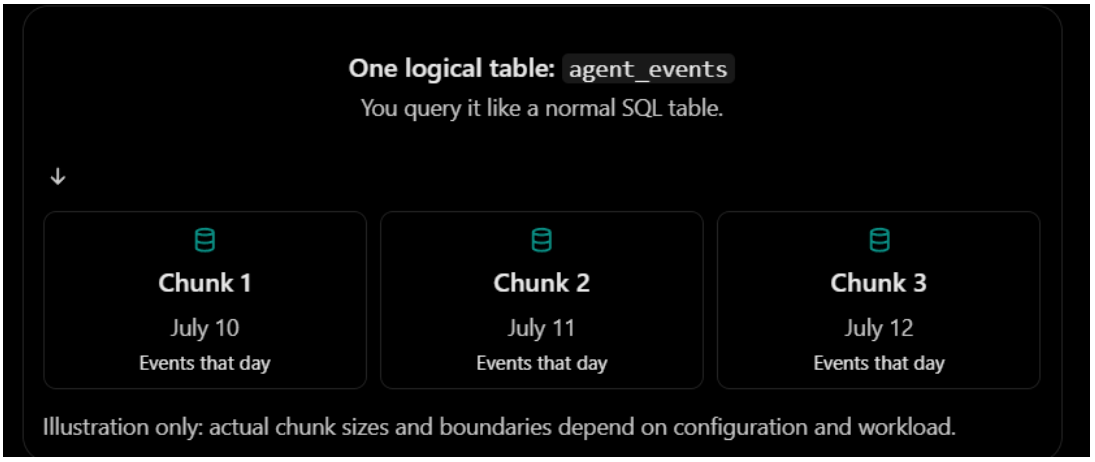
- How long each agent takes.
- How many LLM calls happen.
- How much each review costs.
- When failures or slowdowns occur.

Think of it as an event diary for your system, ordered by time.

2. What is a hypertable?

A normal PostgreSQL table might contain millions of events from many different days, all stored in one logical table.

A hypertable, provided by TimescaleDB (the name “Tiger” in your text refers to its company, Tiger Data), automatically divides the data internally into smaller time-based chunks.



The important part is that you don't manually create a new table for every day. The database manages the chunks for you.

3. Why is this useful?

Imagine you have 500 million agent events accumulated over a year.

Someone asks: “How much did security reviews cost yesterday?”

With time-based partitioning, the database can often skip chunks containing older events and focus on the relevant period. This can make queries faster and make it easier to manage old data.

Other benefits include:

- Retention: Delete or expire old chunks more efficiently.
- Analytics: Calculate average LLM latency or daily token costs.
- Monitoring: Identify spikes in errors or processing times.

Chunking doesn't automatically make every query faster; indexes and query patterns still matter.

4. Do you need a separate time-series database?

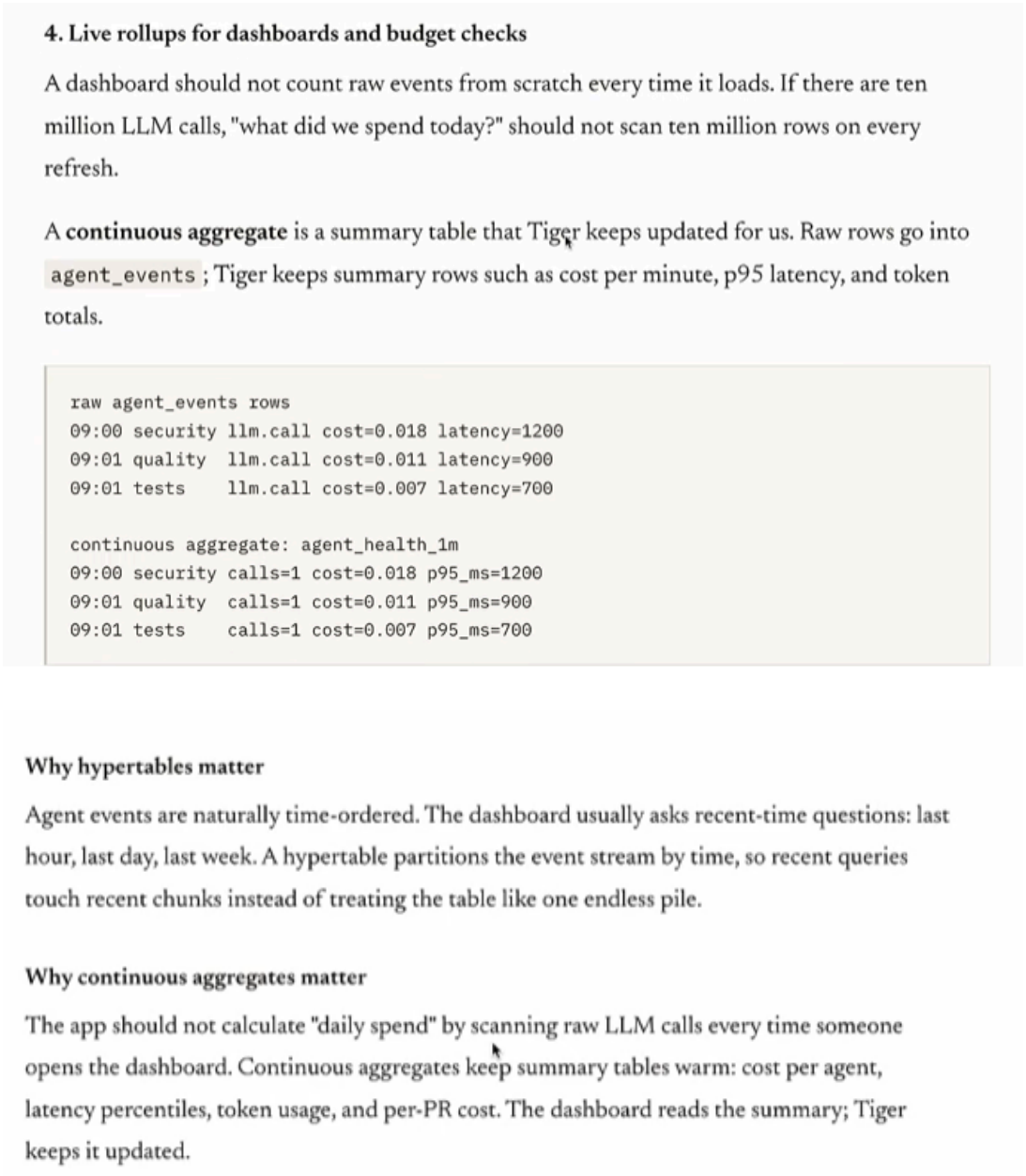
Not necessarily.

For a smaller system, a normal PostgreSQL table with a timestamp column and suitable indexes may be enough. You could introduce TimescaleDB when the volume of events, time-based analytics or retention requirements justify it.

Remember it like this:

- Time-series table: Stores events associated with timestamps.

- Hypertable: Looks like one table but is partitioned into time-based chunks internally.
- TimescaleDB: Extends PostgreSQL with time-series features to manage and query that data efficiently.



4. Live rollups for dashboards and budget checks

A dashboard should not count raw events from scratch every time it loads. If there are ten million LLM calls, "what did we spend today?" should not scan ten million rows on every refresh.

A **continuous aggregate** is a summary table that Tiger keeps updated for us. Raw rows go into `agent_events`; Tiger keeps summary rows such as cost per minute, p95 latency, and token totals.

```
raw agent_events rows
09:00 security llm.call cost=0.018 latency=1200
09:01 quality  llm.call cost=0.011 latency=900
09:01 tests    llm.call cost=0.007 latency=700

continuous aggregate: agent_health_1m
09:00 security calls=1 cost=0.018 p95_ms=1200
09:01 quality  calls=1 cost=0.011 p95_ms=900
09:01 tests    calls=1 cost=0.007 p95_ms=700
```

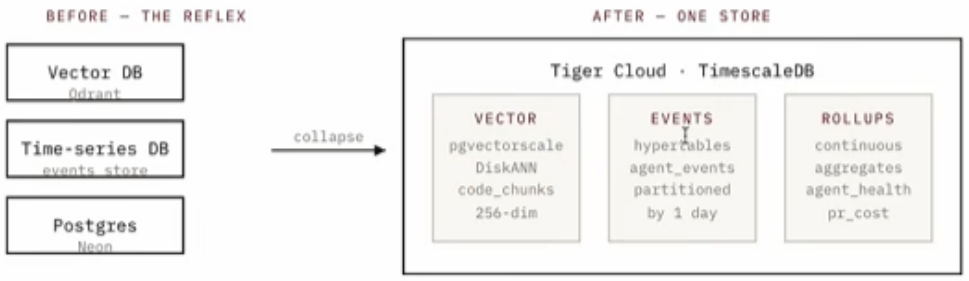
Why hypertables matter

Agent events are naturally time-ordered. The dashboard usually asks recent-time questions: last hour, last day, last week. A hypertable partitions the event stream by time, so recent queries touch recent chunks instead of treating the table like one endless pile.

Why continuous aggregates matter

The app should not calculate "daily spend" by scanning raw LLM calls every time someone opens the dashboard. Continuous aggregates keep summary tables warm: cost per agent, latency percentiles, token usage, and per-PR cost. The dashboard reads the summary; Tiger keeps it updated.

So in conclusion for the above discussed designs:



Redis stays — judgment, not dogma

The job queue is still Redis + ARQ. That is intentional. Queue data is short-lived and high-churn; it does not need vector search, time buckets, or dashboard rollups. "One database" here means one durable data spine, not forcing every workload into SQL.

(Why redis and not rabbitqueue (assuming this isnt a DB and just a free queing service))

Tiger Cloud vs plain Postgres

Plain Postgres is enough for review rows and findings. It is not enough, by itself, for this whole agent. We also need fast vector search over code embeddings and efficient time-series queries over millions of agent events. Tiger Cloud keeps the Postgres programming model but adds TimescaleDB, pgvector, and pgvectorscale.

Tiger Cloud vs Qdrant

Qdrant is good at vector search. The problem is that vector search is not the only question this agent asks. A PR review needs vector similarity plus repo filters, freshness, exact identifier matching, review records, cost records, and audit history. Keeping vectors in Tiger means the retrieval result can live beside the metadata and the review trail.

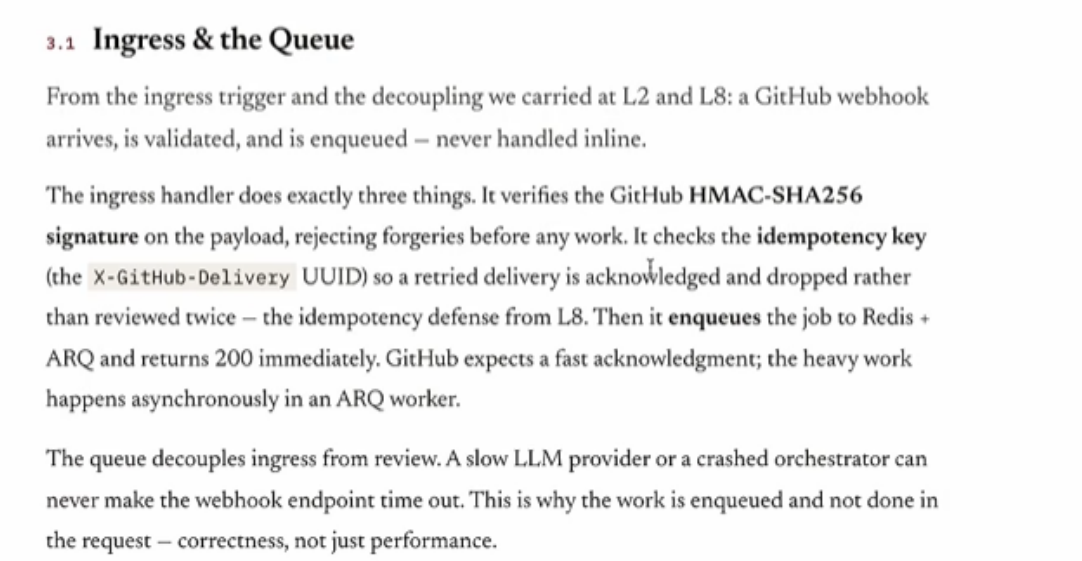
Why DiskANN matters

The simple version: embeddings are lists of numbers, and vector search asks, "which stored lists are closest to this new list?" Many vector indexes keep a lot of their search graph in memory. That gets expensive as the number of code chunks grows. DiskANN is designed to keep more of the index on SSD while still returning close neighbors quickly. That is why pgvectorscale can make Postgres realistic for large vector memory, not just toy demos.

Architecture design steps

Step 1:

Ingress handler inorder to queue the PRs. so Generally github webhooks will take EVERYthing that u send its way, so we need to make sure the agent is only going to consider a PR.



3.1 Ingress & the Queue

From the ingress trigger and the decoupling we carried at L2 and L8: a GitHub webhook arrives, is validated, and is enqueued — never handled inline.

The ingress handler does exactly three things. It verifies the GitHub **HMAC-SHA256 signature** on the payload, rejecting forgeries before any work. It checks the **idempotency key** (the `X-GitHub-Delivery` UUID) so a retried delivery is acknowledged and dropped rather than reviewed twice — the idempotency defense from L8. Then it **enqueues** the job to Redis + ARQ and returns 200 immediately. GitHub expects a fast acknowledgment; the heavy work happens asynchronously in an ARQ worker.

The queue decouples ingress from review. A slow LLM provider or a crashed orchestrator can never make the webhook endpoint time out. This is why the work is enqueued and not done in the request — correctness, not just performance.

To **enqueue** is to put a job onto a queue. The webhook handler doesn't review the pull request. It writes a small message like `{"job": "review_pr", "repo": "...", "pr": 42}` into the queue and immediately returns `200 OK` to GitHub. Because github requires fast acknowledgement (less than 15 secs) but LLM output takes 30 secs so github will start retrying, crashing the process.

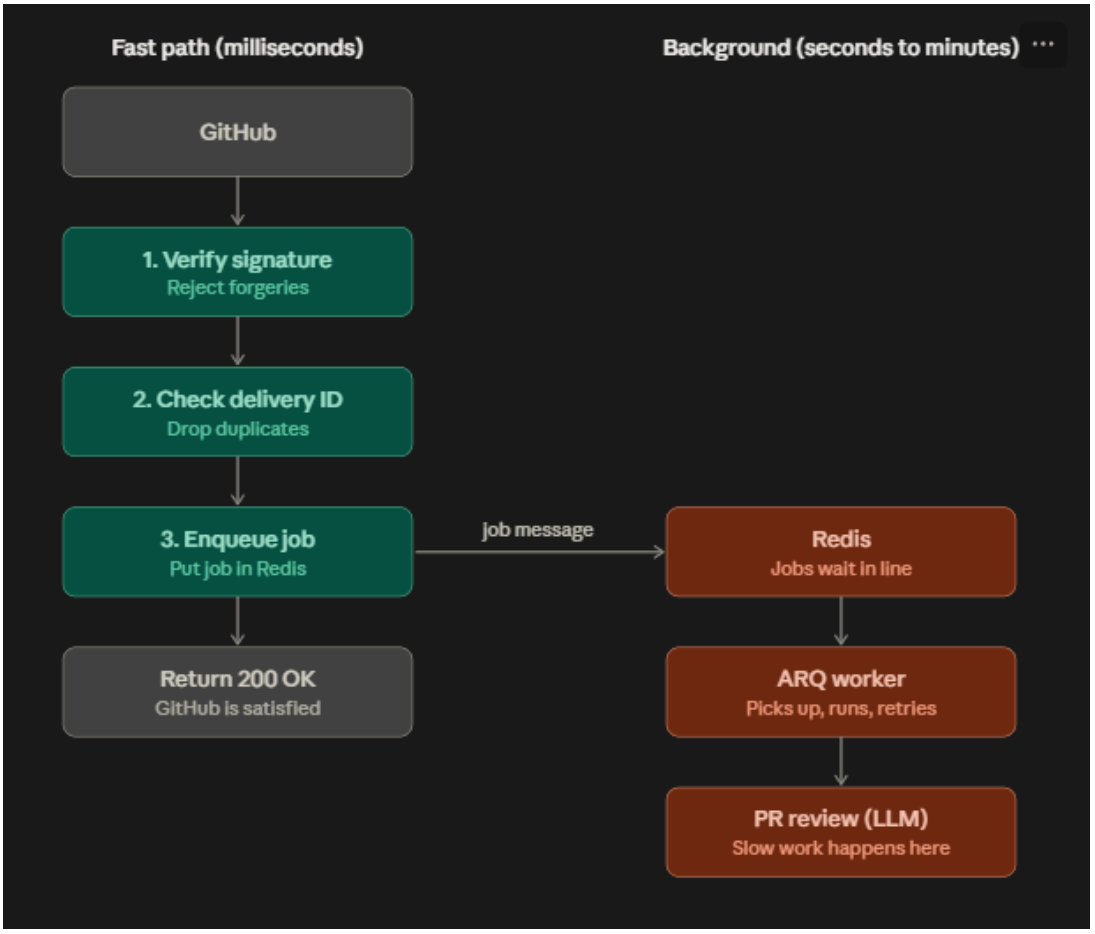
Redis only *stores* the jobs. It's just a place where data sits. It doesn't know how to run anything.

ARQ is the part that does the actual work around it:

- **Worker loop:** keeps watching Redis, pulls a job out, and runs your Python function.

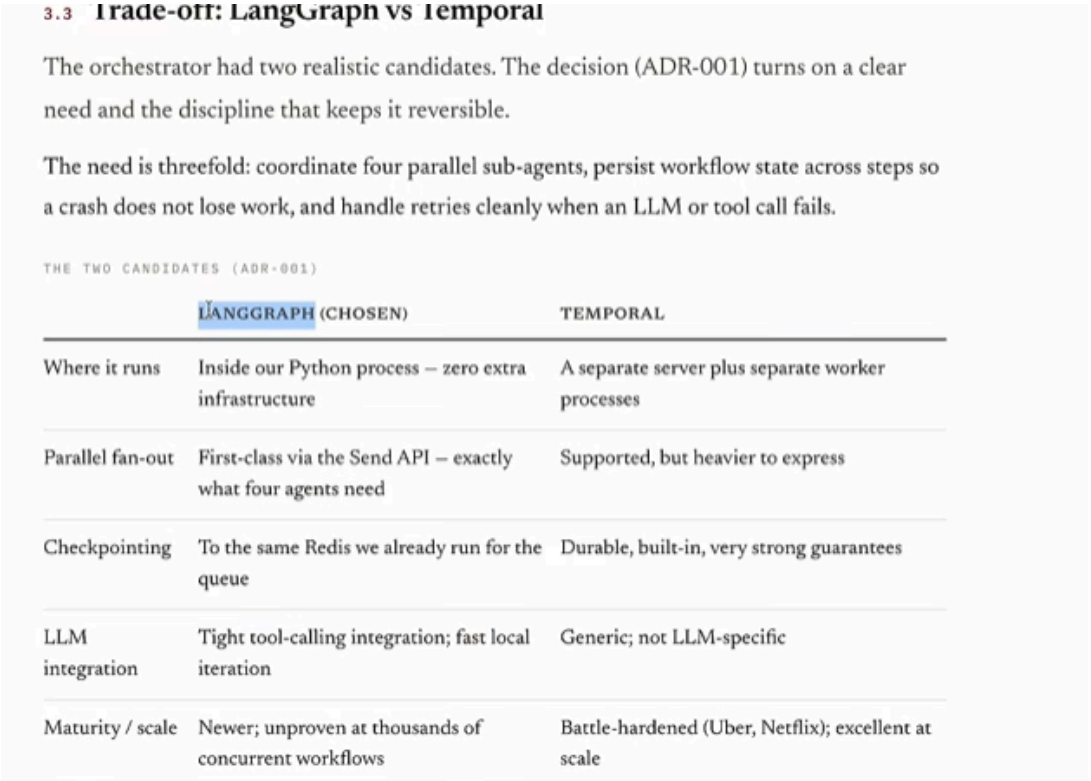
- **Retries and timeouts:** if a job fails, ARQ tries it again, and if it hangs, ARQ cancels it.
- **Job IDs and results:** it tracks which job is which (so your delivery UUID can prevent duplicates) and stores the outcome.

Without ARQ, you'd have to write all of that yourself on top of raw Redis lists. So: **Redis = the mailbox, ARQ = the mail carrier who delivers and handles the job.**



Orchestrator:

Langgraph vs Temporal:



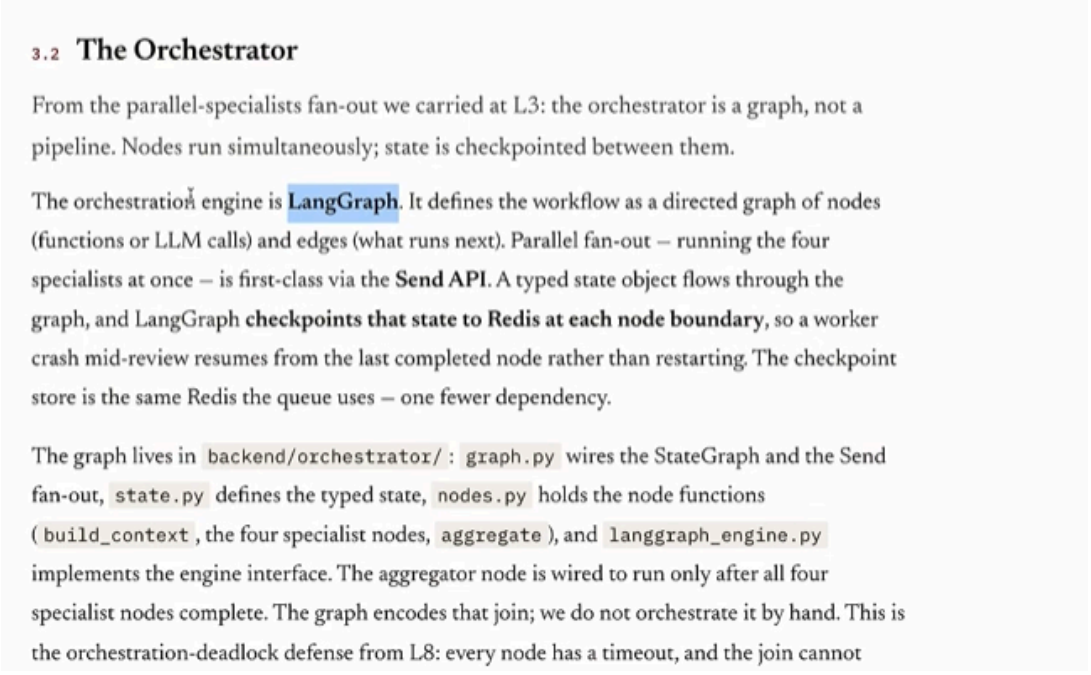
3.3 Trade-off: LangGraph vs Temporal

The orchestrator had two realistic candidates. The decision (ADR-001) turns on a clear need and the discipline that keeps it reversible.

The need is threefold: coordinate four parallel sub-agents, persist workflow state across steps so a crash does not lose work, and handle retries cleanly when an LLM or tool call fails.

THE TWO CANDIDATES (ADR-001)

	LANGGRAPH (CHOSEN)	TEMPORAL
Where it runs	Inside our Python process — zero extra infrastructure	A separate server plus separate worker processes
Parallel fan-out	First-class via the Send API — exactly what four agents need	Supported, but heavier to express
Checkpointing	To the same Redis we already run for the queue	Durable, built-in, very strong guarantees
LLM integration	Tight tool-calling integration; fast local iteration	Generic; not LLM-specific
Maturity / scale	Newer; unproven at thousands of concurrent workflows	Battle-hardened (Uber, Netflix); excellent at scale



3.2 The Orchestrator

From the parallel-specialists fan-out we carried at L3: the orchestrator is a graph, not a pipeline. Nodes run simultaneously; state is checkpointed between them.

The orchestration engine is **LangGraph**. It defines the workflow as a directed graph of nodes (functions or LLM calls) and edges (what runs next). Parallel fan-out — running the four specialists at once — is first-class via the **Send API**. A typed state object flows through the graph, and LangGraph **checkpoints that state to Redis at each node boundary**, so a worker crash mid-review resumes from the last completed node rather than restarting. The checkpoint store is the same Redis the queue uses — one fewer dependency.

The graph lives in `backend/orchestrator/`: `graph.py` wires the StateGraph and the Send fan-out, `state.py` defines the typed state, `nodes.py` holds the node functions (`build_context`, the four specialist nodes, `aggregate`), and `langgraph_engine.py` implements the engine interface. The aggregator node is wired to run only after all four specialist nodes complete. The graph encodes that join; we do not orchestrate it by hand. This is the orchestration-deadlock defense from L8: every node has a timeout, and the join cannot

Why LangGraph here

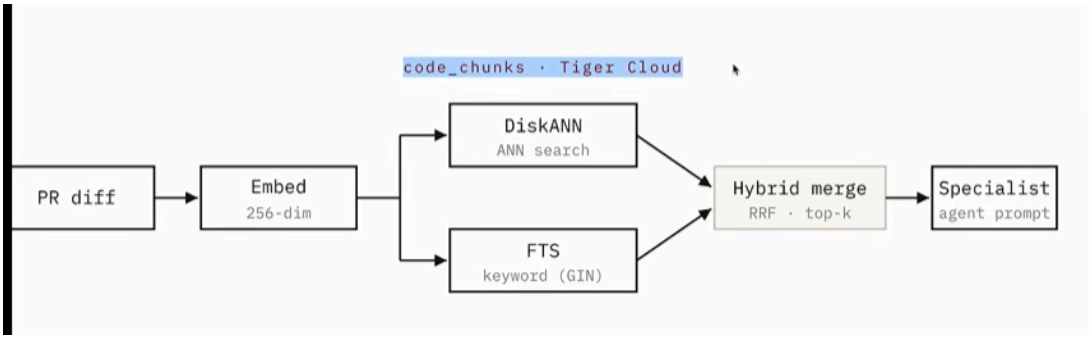
The orchestrator has four needs that LangGraph covers out of the box:

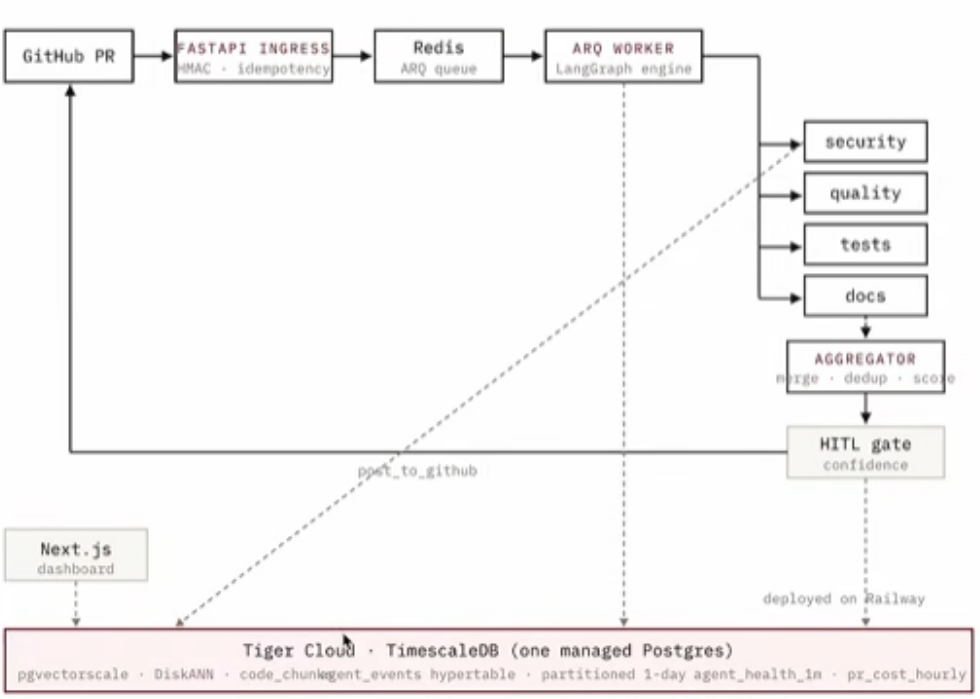
1. **Parallel fan-out and join.** Four specialists (security, quality, tests, docs) run at once, and the aggregator runs only after all four finish. LangGraph's `Send`

API and graph edges express that directly, so you don’t hand-write the “wait for all four” logic.

2. **Checkpointing.** State is saved to Redis after each node. If a worker crashes after 3 of 4 specialists finish, the job resumes from there instead of paying for those LLM calls again. Building this yourself is the hardest part.
3. **Typed shared state.** One state object flows through the graph, so every node reads and writes a known schema.
4. **Human-in-the-loop.** LangGraph can pause a graph at a node, wait for a human, and resume, which matches your “Confident? No → route to human” gate.

Retrieval layer:





TimescaleDB is **not a separate database**. It's an extension that runs inside PostgreSQL, like pgvector does for vectors.

- **PostgreSQL** is the regular database: tables, SQL, JOINs, transactions.
- **TimescaleDB** adds features on top for timestamped data (logs, events, metrics): hypertables (auto-split by time), continuous aggregates (auto-updated summaries), and easy deletion or compression of old data.

You still write normal SQL, still use the same Postgres connection, and can JOIN `agent_events` with your other tables. So you get time-series speed without running a second database system, which fits your notes' rule of starting with the fewest storage systems.

For a small event volume, a plain Postgres table would work. TimescaleDB pays off when events grow into the millions.